

Tribhuvan University

Institute of Science and Technology



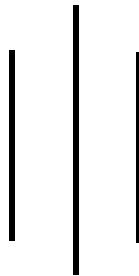
A Term Paper

On

**“Evaluation and Context Optimization of Retrieval-Augmented Generation for
Document Question Answering”**

Submitted To

Central Department of Computer Science and Information Technology
Tribhuvan University, Kirtipur
Kathmandu, Nepal



Submitted By

Melina Giri
Roll No: 8125018

April, 2026

**In partial fulfillment of the requirement for Master’s Degree in Information
Technology (MIT), Third Semester**



Tribhuvan University
Institute of Science and Technology

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this term paper, prepared under the supervision of Asst. Prof. Balkrishna Subedi entitled "**Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering**" be accepted as fulfillment in partial requirement for the degree of Master's in Information Technology. To the best of my knowledge, this is an original work in computer science.

.....

Asst. Prof. Balkrishna Subedi

(Supervisor)

Central Department of Computer Science and Information Technology

LETTER OF APPROVAL

This is to certify that the term paper prepared by **Ms. Melina Giri** entitled "**Evaluation and Context Optimization of Retrieval-Augmented Generation for Document Question Answering**" in partial fulfillment of the requirements for the degree of Masters in Information Technology has been well studied. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

Evaluation Committee

.....
Asst. Prof. Dr. Dhiraj Kedar Pandey
(H.O.D)

Central Department of Computer Science and
Information Technology

.....
Asst. Prof. Balkrishna Subedi
(Supervisor)

Central Department of Computer Science and
Information Technology

.....
(Internal)

ACKNOWLEDGEMENT

The success and final outcome of this report required a lot of guidance and assistance from many people and I am very fortunate to have received it throughout the completion of this work. I am very glad to express my deepest sense of gratitude and sincere thanks to my highly respected and esteemed supervisor **Asst. Prof. Balkrishna Subedi**, Central Department of Computer Science and Information Technology, for his valuable supervision, guidance, encouragement, and support in completing this paper.

I am also thankful to **Asst. Prof. Dr. Dhiraj Kedar Pandey**, HOD of Central Department of Computer Science and Information Technology for his constant support throughout the period. Furthermore, with immense pleasure, I extend my deepest gratitude to the Department of Computer Science and Information Technology, Tribhuvan University, and all the faculty members of CDCSIT for providing the platform to explore the knowledge of interest.

I would also like to acknowledge the authors and contributors of the **Open RAG Benchmark** provided by Vectara. The publicly available benchmark dataset was fundamental for conducting the retrieval and generation experiments presented in this research.

At the end I would like to express my sincere thanks to all my friends and others who helped me directly or indirectly.

Melina Giri (8125018)

Central Department of Computer Science and Information Technology

ABSTRACT

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation to provide answers using relevant information retrieved from external document collections. This research implements and evaluates a **RAG-based document question-answering system** using the **Vectara Open RAG Benchmark**. The system loads the benchmark's pre-segmented document sections directly, generates dense embeddings using the **all-MiniLM-L12-v2** model, and retrieves relevant sections using vector similarity. Retrieval performance was evaluated using **3,045 benchmark queries** at retrieval depths of $k=3, 5, 10$, and 20 using Hit@ k , Mean Reciprocal Rank (MRR), and normalized Discounted Cumulative Gain (nDCG). The results show that retrieval effectiveness improved with increasing retrieval depth, with Hit@20 reaching **82.33%**, compared with **61.18%** at $k=3$. Generation performance was evaluated using **60 questions** under three configurations: **R0_k3**, **R1_k10**, and **R2_k10_optimized**, using the locally executed **Qwen2.5-3B** language model through Ollama. Among the three configurations, **R1_k10** achieved the **highest mean lexical F1 score** of 0.3136, while **R2_k10_optimized** achieved the **lowest average generation latency** of 12.098 seconds. A paired Wilcoxon signed-rank test on the 60 per-question lexical F1 scores showed that the decrease in F1 from R1_k10 to R2_k10_optimized was statistically significant ($p = 0.005$), while the differences involving R0_k3 were not significant. The findings indicate that increasing retrieval depth improves retrieval coverage, and that context optimization measurably improves generation efficiency but comes at the cost of a statistically significant, if small, reduction in lexical answer overlap. The research also identifies the limitations of lexical F1 and recommends semantic and human-based evaluation methods for future research.

Keywords: Retrieval-Augmented Generation, Document Question Answering, Dense Retrieval, Sentence Embeddings, Vector Database, Context Optimization, Open RAG Benchmark, Qwen2.5.

Table of Contents

SUPERVISOR’S RECOMMENDATION	i
LETTER OF APPROVAL	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Report Organization	3
CHAPTER 2: PREVIOUS WORKS, DISCUSSIONS AND FINDINGS	4
2.1 Background Study	4
2.2 Literature Review	4
CHAPTER 3: METHODOLOGY AND CRITICAL ANALYSIS	6
3.1. Methodology	6
3.1.1. Flowchart	6
3.1.2. Dataset Description	7
3.1.3. Retrieval Analysis	8
3.1.4. Algorithm Description	9
3.2. Implementation	10
3.2.1. Implementation Tools	10
3.2.2. Implementation Details	11
CHAPTER 4: RESULTS/ FINDINGS	12
4.1. Findings and Observations	12
4.2. Result Analysis	12
4.2.1. Quantitative Analysis	12
4.2.2. Qualitative Analysis (Actual System Output)	16
CHAPTER 5: CONCLUSION	18
5.1. Conclusion	18
5.2. Future Work	18
References	20
Appendix	21

A.1 Main Interface Code	21
A.2 Loading Documents and Corpus Sections	22
A.3 Embedding Generation and Vector Retrieval	23
A.4 RAG Answer Generation Code	24
A.5 Evaluation and Result Analysis Code	25

LIST OF FIGURES

Figure 3.1 : Overview of RAG-Based Document Question-Answering Workflow	6
Figure 4.1.1 : Retrieval Performance at Different Retrieval Depths	13
Figure 4.1.2 : Lexical F1 Comparison of RAG Generation Configurations	14
Figure 4.1.3 : Average Generation Latency Across RAG Configurations	14
Figure 4.1.4 : Sample Question and Generated Answer from the RAG System	16
Figure 4.1.5 : RAG Pipeline	16

LIST OF TABLES

Table 4.1 : Retrieval Performance at Different Retrieval Depths	12
Table 4.1.2 : Generation Performance of Different RAG Configurations	15
Table 4.1.3 : Paired Lexical-F1 Differences and Wilcoxon Signed-Rank Test Results	15

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
F1	F1 Score
IDK	I Don't Know
k	Retrieval Depth / Number of Retrieved Sections
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	normalized Discounted Cumulative Gain
PDF	Portable Document Format
QA	Question Answering
RAG	Retrieval-Augmented Generation

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid growth of digital documents and research publications has created a significant challenge in efficiently finding and extracting relevant information. Traditional document search systems generally rely on keyword matching, which may return large numbers of documents without providing a direct, context-aware answer to the user's question. As the volume of digital information continues to increase, there is a growing need for systems that can retrieve relevant information and generate meaningful answers from that information.

Recent advances in **Large Language Models (LLMs)** have significantly improved natural language question-answering capabilities. However, language models may have limitations when answering knowledge-intensive questions that require information from external document collections. This limitation has motivated the development of **Retrieval-Augmented Generation (RAG)** systems [1].

RAG combines **information retrieval** with **language generation**. Instead of relying only on information encoded in the language model, a RAG system first retrieves relevant sections from an external document collection and provides those sections as context to the language model. The model subsequently generates an answer using the retrieved context [1]. A typical RAG pipeline consists of document processing, section retrieval, embedding generation, vector-based retrieval, context selection, and answer generation [1].

The effectiveness of a RAG system depends not only on the quality of the language model but also on the quality and quantity of retrieved context. Retrieving too few sections may omit information necessary to answer a question, whereas retrieving too many sections may introduce redundant or irrelevant information and increase computational cost. Therefore, determining an appropriate retrieval depth and optimizing the retrieved context are important considerations in RAG-based document question answering.

Research Gap: Although Retrieval-Augmented Generation systems commonly use retrieval depth and contextual information to support answer generation, the combined effects of retrieval depth and context optimization on retrieval effectiveness, answer quality, and generation efficiency require further empirical evaluation. In particular, there is a need to compare different retrieval depths and context-selection strategies within a standardized document question-answering benchmark and to examine whether differences in generated-answer performance are statistically significant. This research addresses this gap by evaluating retrieval at different depths and comparing top-3, top-10, and optimized top-10 retrieval configurations.

1.2 Problem Statement

Retrieval-Augmented Generation (RAG) systems depend on the retrieval of relevant document sections to provide context for answer generation. However, the choice of retrieval depth can affect both the relevance of the retrieved context and the amount of information passed to the language model. A smaller retrieval depth may omit relevant information, whereas a larger retrieval depth may introduce redundant context and increase the amount of information processed during generation.

Context optimization provides another approach to controlling the retrieved information by reducing redundant sections before generation. However, the effect of combining different retrieval depths with context optimization needs to be evaluated in terms of both answer quality and generation efficiency. In particular, it is necessary to determine how top-3 retrieval, top-10 retrieval, and optimized top-10 retrieval differ in retrieval effectiveness, generated-answer quality, context size, and generation latency.

Therefore, the problem addressed in this research is the **lack of a systematic experimental comparison of retrieval depth and context optimization within a common RAG-based document question-answering framework**. The research evaluates retrieval at $k=3,5,10$, and 20 using $\text{Hit}@k$, Mean Reciprocal Rank (MRR), and nDCG, and compares three generation configurations using lexical F1 and generation-efficiency measures. Statistical significance testing is also used to examine whether observed differences in answer quality between configurations are statistically meaningful.

1.3 Objectives

To evaluate the performance of a Retrieval-Augmented Generation (RAG) system for document question answering and analyze the effects of retrieval depth and context optimization on answer quality and generation efficiency.

The specific objectives of this research are:

- To implement a document question-answering pipeline using Retrieval-Augmented Generation, incorporating document processing, embedding generation, vector-based retrieval, and language-model-based answer generation.
- To evaluate and compare retrieval and generation performance at different retrieval depths and RAG configurations using $\text{Hit}@k$, Mean Reciprocal Rank (MRR), nDCG, and lexical F1 metrics, including statistical significance testing.
- To analyze the effect of context optimization on answer quality and generation efficiency, particularly in terms of context size and generation latency, and identify the strengths and limitations of the evaluated RAG configurations.

1.4 Report Organization

This term paper is organized into five chapters, presenting the research in a systematic and structured manner.

Chapter 1: Introduction presents the background of Retrieval-Augmented Generation (RAG) and document question answering, the problem statement, research objectives, and report organization.

Chapter 2: Background Study and Literature Review discusses the theoretical concepts and previous studies related to Large Language Models (LLMs), RAG, dense retrieval, sentence embeddings, context optimization, and evaluation methods. It also identifies the research gap.

Chapter 3: Methodology and Implementation describes the dataset, preprocessing, embedding generation, vector retrieval, cosine similarity, retrieval depths, evaluation metrics, RAG configurations, and context optimization approach. It also presents the tools and models used in the implementation.

Chapter 4: Results and Findings presents and analyzes the experimental results using retrieval and generation evaluation metrics. It compares the RAG configurations in terms of retrieval effectiveness, answer quality, latency, context size, and statistical significance.

Chapter 5: Conclusion and Future Work summarizes the major findings and conclusions and discusses possible future improvements, including context optimization, different models and embeddings, enhanced evaluation methods, and real-world applications.

The **References** section lists the sources consulted, while the **Appendix** provides relevant implementation and evaluation code.

CHAPTER 2: PREVIOUS WORKS, DISCUSSIONS AND FINDINGS

2.1 Background Study

The rapid growth of digital documents and research publications has created challenges in efficiently finding and extracting relevant information. Traditional document retrieval systems mainly rely on keyword matching, requiring users to examine multiple documents to find relevant information. As digital information continues to increase, effective document question-answering systems have become increasingly important.

Early document question-answering approaches used **keyword search, rule-based methods, and information retrieval techniques**. Although these methods can identify documents containing specific terms, they may struggle to understand the semantic meaning of questions or generate direct answers from multiple relevant passages.

The development of **Large Language Models (LLMs)** has improved natural-language question answering. However, LLMs may have limited access to information contained in specific external document collections, leading to the development of **Retrieval-Augmented Generation (RAG)** systems [1].

RAG combines **information retrieval with language generation**. Relevant information is first retrieved from an external document collection and then provided as context to a language model, which generates an answer based on the retrieved information [1]. A typical RAG system involves document processing, embedding generation, vector storage, similarity-based retrieval, context selection, and answer generation.

RAG performance depends strongly on the quality and amount of retrieved context. Retrieving too little information may omit important evidence, whereas retrieving too much may introduce redundancy and increase computational cost. Therefore, **retrieval depth and context optimization** are important for efficient document question answering.

2.2 Literature Review

The increasing availability of digital documents has made efficient document retrieval and question answering an important research area. Traditional information retrieval methods generally rely on lexical matching between queries and documents. Although effective for exact keyword searches, they may fail when queries and relevant documents use different words with similar meanings.

The introduction of **dense vector representations and embeddings** has enabled semantic information retrieval. Text can be converted into numerical vectors, allowing semantically

similar queries and document sections to be identified using similarity measures [2][3]. **Cosine similarity** is commonly used to measure similarity between vector representations.

Recent developments in **Large Language Models** have improved natural-language generation. However, LLM-only systems may suffer from hallucination, outdated knowledge, and limited access to domain-specific documents. **RAG** addresses these limitations by retrieving external evidence before generating an answer, providing the model with relevant document context [1].

An important factor in RAG performance is the **number of retrieved documents or sections**, commonly represented by k . A small k may provide insufficient context, whereas a large k may introduce irrelevant or repetitive information. Context optimization can therefore be used to remove redundant information before generation.

RAG systems are commonly evaluated at two levels: **retrieval performance** and **generation performance**. Retrieval can be evaluated using **Hit@k**, **Mean Reciprocal Rank (MRR)**, and **normalized Discounted Cumulative Gain (nDCG)**. Generated answers can be evaluated using metrics such as lexical F1, although lexical measures primarily assess word overlap rather than complete semantic correctness. When comparing metrics across multiple system configurations using the same questions, a paired statistical test such as the **Wilcoxon signed-rank test** can determine whether observed differences are systematic rather than due to sampling variation.

In conclusion, previous research indicates that effective RAG performance depends on both **retrieval quality and generation context**. This foundation supports the present research, which evaluates different retrieval depths and context configurations using answer quality, generation efficiency, and paired significance testing.

CHAPTER 3: METHODOLOGY AND CRITICAL ANALYSIS

3.1. Methodology

3.1.1. Flowchart

The overall workflow of the proposed RAG-based document question-answering system consists of document preparation, embedding generation, vector retrieval, context selection, and answer generation.

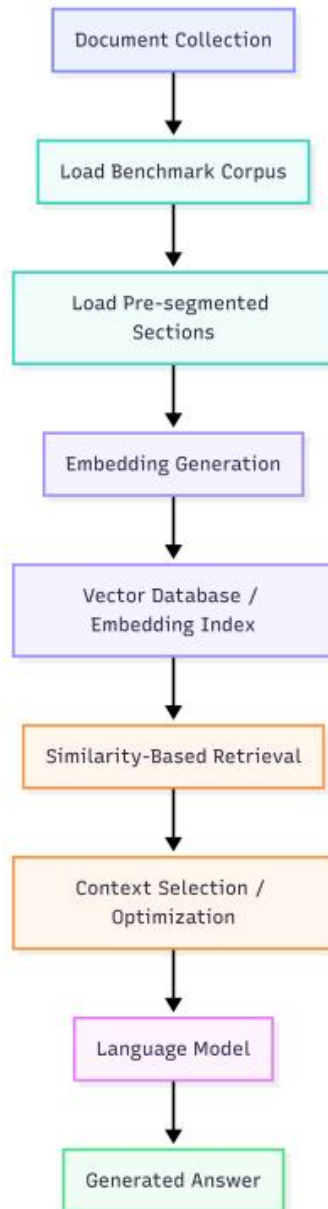


Figure 3.1: Overview of RAG-Based Document Question-Answering Workflow

The research process followed a sequential RAG-based document question-answering workflow. First, the Vectara Open RAG Benchmark was loaded, and the pre-segmented document sections and their metadata were prepared for retrieval. The document sections were then converted into 384-dimensional dense embeddings using the all-MiniLM-L12-v2 model and stored for

similarity-based retrieval. For each question, the query was converted into the same embedding space, and cosine similarity was calculated between the query and candidate document sections. The sections were ranked according to their similarity scores, and the top- k sections were retrieved. Retrieval performance was evaluated at $k=3,5,10$, and 20 using Hit@ k , MRR, and nDCG on the benchmark queries.

For generation evaluation, 60 questions were selected from the benchmark and evaluated using three retrieval-context configurations: top-3 retrieval (R0_k3), top-10 retrieval (R1_k10), and optimized top-10 retrieval (R2_k10_optimized). For R2_k10_optimized, redundant sections were removed using Jaccard word-set overlap, with an overlap threshold of 0.70 and a maximum of five sections retained. The selected context was then truncated to a maximum of 8,000 characters for all three configurations. The question and retrieved context were provided to the locally executed Qwen2.5:3B language model, which generated the final answer. The generated answers were compared with the benchmark reference answers using lexical F1, while context size and generation latency were recorded to assess generation efficiency. Finally, paired Wilcoxon signed-rank tests were applied to the 60 per-question F1 scores to examine differences between the generation configurations.

3.1.2. Dataset Description

The dataset used for this research is the **Vectara Open RAG Benchmark**, which is designed for evaluating retrieval-augmented generation systems on research-paper documents. The benchmark contains **1,000 PDF papers and 3,045 valid question-answer pairs**. The benchmark includes both abstractive and extractive questions and contains text, text-image, text-table, and text-table-image source categories.

Data Processing

Before performing retrieval, the document corpus is loaded and transformed into searchable sections and numerical vector representations. The main processing steps include:

1. Document Preprocessing

- **PDF processing:** Each research paper in the Open RAG Benchmark corpus is provided as a structured JSON record containing document metadata and a list of pre-segmented sections.
- **Text segmentation:** The Open RAG Benchmark provides documents as pre-segmented sections. The research used these benchmark-provided sections directly as retrieval units, without applying additional token- or character-based splitting.

Each section was retained together with its document and section identifier for subsequent embedding and retrieval evaluation.

- **Metadata preparation:** Document identifiers and section identifiers were retained to match retrieved sections against the benchmark's relevance judgments.
- **Embedding generation:** Each document section was converted into a numerical vector representation.

2. Embedding Generation

- The research uses the **all-MiniLM-L12-v2** sentence-transformer model to generate dense embeddings for document sections and queries [3][6].
- Each document section is represented as a **384-dimensional vector**. These vectors allow semantic similarity to be calculated between a user query and **candidate document sections** [6].

3.1.3. Retrieval Analysis

The retrieval stage uses dense-vector similarity to identify the most relevant document sections for a given question, following the general principle of dense retrieval [2]. The query is converted into the same embedding space as the document sections, and similarity scores are calculated.

Different retrieval depths were evaluated, with $k \in \{3,5,10,20\}$. The retrieved sections were then compared against the benchmark's relevant sections.

Cosine Similarity (Mathematical Formula)

The similarity between a query vector q and document vector d is calculated using:

$$\text{Similarity}(q,d) = \frac{q \cdot d}{\|q\| \cdot \|d\|} \quad (1)$$

Where:

- q = query embedding vector
- d = document-section embedding vector
- $q \cdot d$ = dot product between the two vectors
- $\|q\|$ = magnitude of query vector
- $\|d\|$ = magnitude of document vector

Retrieval Evaluation Metrics

- **Hit@k:** Measures whether at least one relevant document section appears within the top k retrieved results.

$$\text{Hit}@k = \begin{cases} 1, & \text{if at least one relevant section is in top } k \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

- **Mean Reciprocal Rank (MRR):** Measures how highly the first relevant result is ranked.

$$\text{MRR} = \frac{1}{N} \sum_{i=0}^N \frac{1}{\text{rank}_i} \quad (3)$$

where rank_i is the position of the first relevant retrieved section for query ii .

- **nDCG:** Measures ranking quality while giving greater importance to highly ranked relevant results.

$$\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} \quad (4)$$

where DCG is the discounted cumulative gain and IDCG is the ideal DCG.

3.1.4. Algorithm Description

The algorithm aims to answer document-based questions using Retrieval-Augmented Generation (RAG) [1]. It involves loading pre-segmented document sections, generating embeddings, retrieving relevant sections, optionally optimizing the retrieved context, and generating an answer using a language model.

Algorithm 1: Retrieval-Augmented Generation for Document Question Answering

Input: Document corpus D , user question q , retrieval depth k

Output: Generated answer A

Steps:

1. Document Processing

- Load the document corpus.
- Load the pre-segmented document sections provided by the Open RAG Benchmark.
- Generate embeddings for each document section.

2. Query Processing and Retrieval

- Accept the question from the user.
- Generate an embedding representation of the question.
- Calculate similarity between the query embedding and document-section embeddings.
- Rank document sections according to similarity.
- Select the top- k sections.

3. Context Optimization (R2_k10_optimized only)

- Examine the top-10 retrieved sections in ranked order.
- Convert each section into a set of lowercased words.

- Calculate the Jaccard word-set overlap between the candidate section and each previously selected section: $\text{overlap} = |A \cap B| / |A \cup B|$.
- Mark the candidate section as redundant if its overlap with any previously selected section is ≥ 0.70 .
- Remove redundant sections and retain a maximum of five non-redundant sections.

4. Context Assembly

- Concatenate the selected sections and truncate the combined context to a maximum of 8,000 characters, applied uniformly across all three configurations (R0_k3, R1_k10, R2_k10_optimized).

5. Generate Answer

- Combine the question with the retrieved context.
- Provide the context and question to the language model.
- Generate the final answer.
- If the available context is insufficient, the system is instructed to respond that the information is unknown.

3.2. Implementation

3.2.1. Implementation Tools

The implementation was carried out using the tools below for document processing, retrieval, vector representation, and answer generation:

- **Python 3.x** – Main programming language
- **FastAPI** – Backend API framework
- **Streamlit** – User-interface/frontend framework
- **LangChain** – RAG pipeline and language-model integration
- **Sentence Transformers** – Generation of dense text embeddings (all-MiniLM-L12-v2)
- **NumPy** – Vector similarity computation for retrieval
- **Ollama** – Local language-model execution
- **Qwen2.5:3B** – Local language model used for the final generation experiment
- **Pandas** – Dataset processing and result analysis
- **SciPy** – Paired t-test and Wilcoxon signed-rank significance testing
- **Matplotlib** – Visualization of experimental results
- **VS Code** – Development environment
- **Dataset** – Vectara Open RAG Benchmark

3.2.2. Implementation Details

The RAG system converts document sections and user queries into dense numerical representations using the all-MiniLM-L12-v2 embedding model. Each representation contains 384 dimensions. Query and document vectors are compared using cosine similarity to identify the most relevant document sections.

The retrieval experiment was performed using 3,045 benchmark queries. Retrieval performance was evaluated at four different retrieval depths: $k=3$, 5, 10, and 20.

For generation evaluation, 60 questions were selected from the benchmark. Three retrieval-context configurations were evaluated:

- **R0_k3**: Top 3 retrieved sections were provided to the language model.
- **R1_k10**: Top 10 retrieved sections were provided to the language model.
- **R2_k10_optimized**: Top 10 sections were retrieved and redundant context was removed (Jaccard overlap ≥ 0.70 , maximum 5 sections retained) before generation.

In all three configurations, the assembled context was additionally truncated to a maximum of 8,000 characters before being passed to the language model, which explains why the mean context size of R1_k10 in Table 2.2 reaches the 8,000-character ceiling.

The final generation experiment used the Qwen2.5:3B language model, executed locally through Ollama [5]. Local execution eliminated dependence on an external API daily-token quota during the final experiment.

The generated answers were compared with the benchmark reference answers using token-set lexical F1, computed as the harmonic mean of precision and recall between the lowercased alphanumeric token sets of the generated answer and the gold answer. Generation latency and context size were also recorded to evaluate efficiency. To determine whether differences in mean lexical F1 between configurations were statistically meaningful, a paired Wilcoxon signed-rank test was applied to the 60 per-question F1 scores for each pairwise comparison, since the same 60 questions were evaluated under all three configurations.

CHAPTER 4: RESULTS/ FINDINGS

4.1. Findings and Observations

The experimental evaluation of the Retrieval-Augmented Generation (RAG) system was conducted to address the research objectives by examining retrieval performance at different retrieval depths and comparing RAG configurations in terms of answer quality and generation efficiency. The major findings and observations are summarized below:

- **Retrieval Performance:** Retrieval effectiveness improved consistently as the retrieval depth increased from $k=3$ to $k=20$. Hit@ k increased from 61.18% at $k=3$ to 82.33% at $k=20$, while MRR and nDCG also showed improvement.
- **Generation Performance:** Among the three configurations, R1_k10 achieved the highest mean lexical F1 score of 0.3136, compared with 0.3030 for R0_k3 and 0.3046 for R2_k10_optimized.
- **Generation Efficiency:** The R2_k10_optimized configuration achieved the lowest mean generation latency of 12.098 seconds, compared with 16.958 seconds for R1_k10. This shows that removing redundant context can reduce generation time.
- **Effect of Context Optimization:** Context optimization produced a small reduction in lexical F1 from 0.3136 to 0.3046 compared with the unoptimized top-10 configuration. The difference was statistically significant according to the Wilcoxon signed-rank test ($p = 0.005$).
- **Overall Observation:** The results indicate that increasing retrieval depth improved retrieval effectiveness, while context optimization reduced generation latency by reducing redundant context. However, the efficiency improvement was accompanied by a small reduction in lexical answer overlap.

4.2. Result Analysis

4.2.1. Quantitative Analysis

Retrieval Performance at Different k Values

Table 4.1: Retrieval Performance at Different Retrieval Depths

Retrieval Depth	Hit@ k	MRR	nDCG
$k = 3$	0.6118	0.5061	0.5333
$k = 5$	0.6893	0.5239	0.5653
$k = 10$	0.7718	0.5349	0.5920
$k = 20$	0.8233	0.5385	0.6051

The retrieval results demonstrate that increasing the retrieval depth consistently improved retrieval performance.

- At $k=3$, Hit@3 was 61.18%.
- At $k=5$, Hit@5 increased to 68.93%.
- At $k=10$, Hit@10 increased to 77.18%.
- At $k=20$, Hit@20 reached 82.33%.

Similarly, MRR increased from 0.5061 at $k=3$ to 0.5385 at $k=20$, while nDCG increased from 0.5333 to 0.6051.

These results indicate that increasing retrieval depth improves the probability of retrieving relevant information and also improves the overall ranking quality.

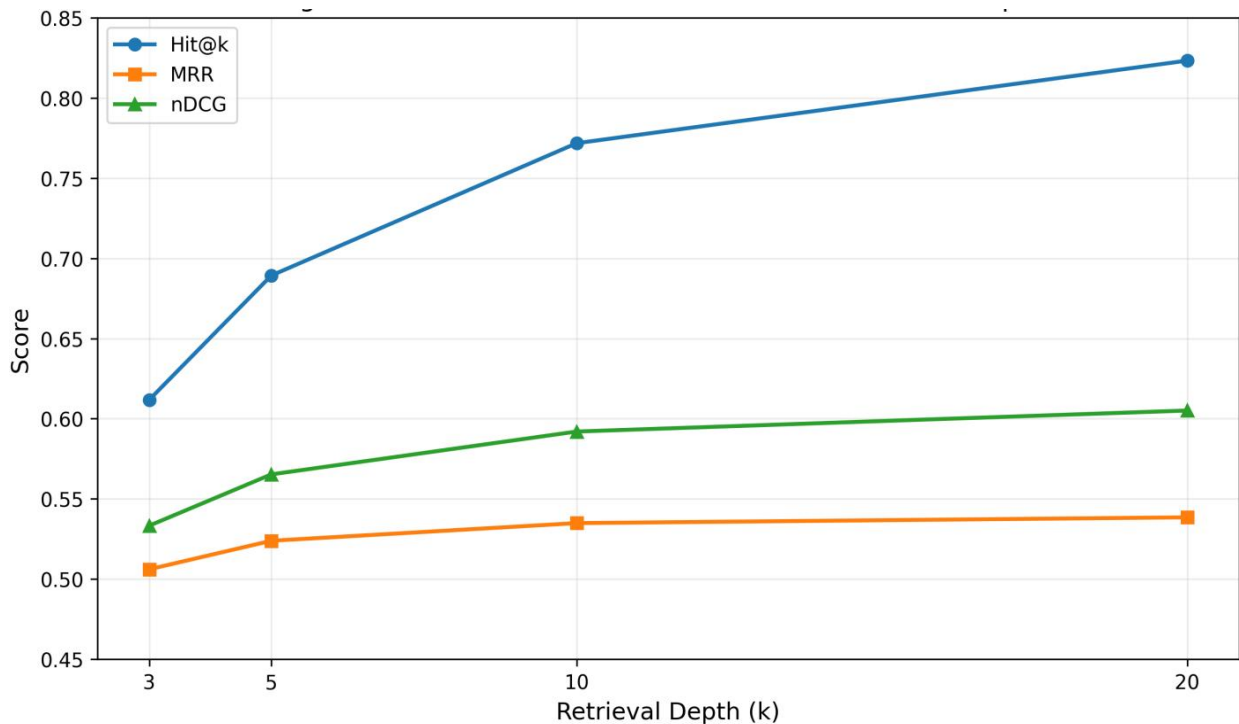


Figure 4.1.1: Retrieval Performance at Different Retrieval Depths

Generation Latency Analysis

The generation latency showed a substantial difference between the three configurations.

- **R0_k3:** 185.600 seconds
- **R1_k10:** 16.958 seconds
- **R2_k10_optimized:** 12.098 seconds

The optimized configuration achieved the lowest average latency. Compared with R1_k10, R2_k10_optimized reduced average latency by approximately 4.86 seconds. The substantially higher latency observed for R0_k3 suggests that generation latency may have been influenced by the local inference environment and runtime behavior in addition to context size. Therefore, the latency results should be interpreted with caution.

In the comparison between R1_k10 and R2_k10_optimized, the reduction in context was accompanied by lower generation latency.

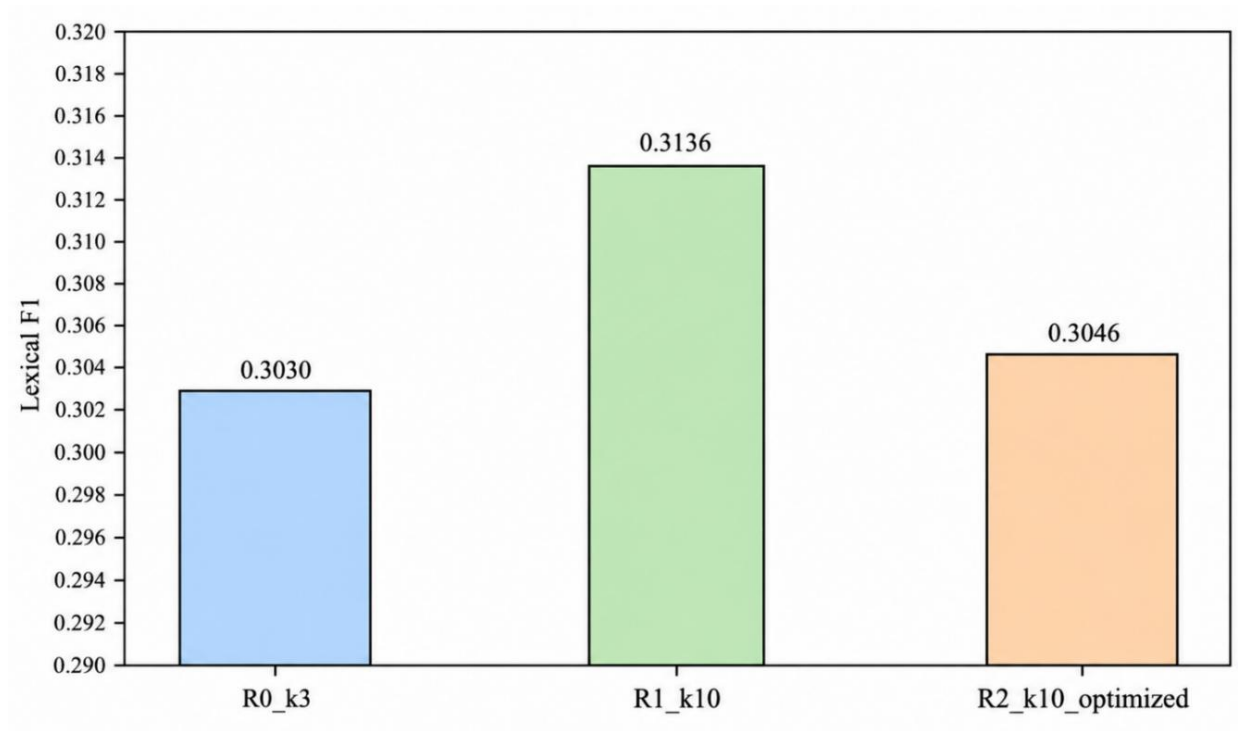


Figure 4.1.2: Lexical F1 Comparison of RAG Generation Configurations

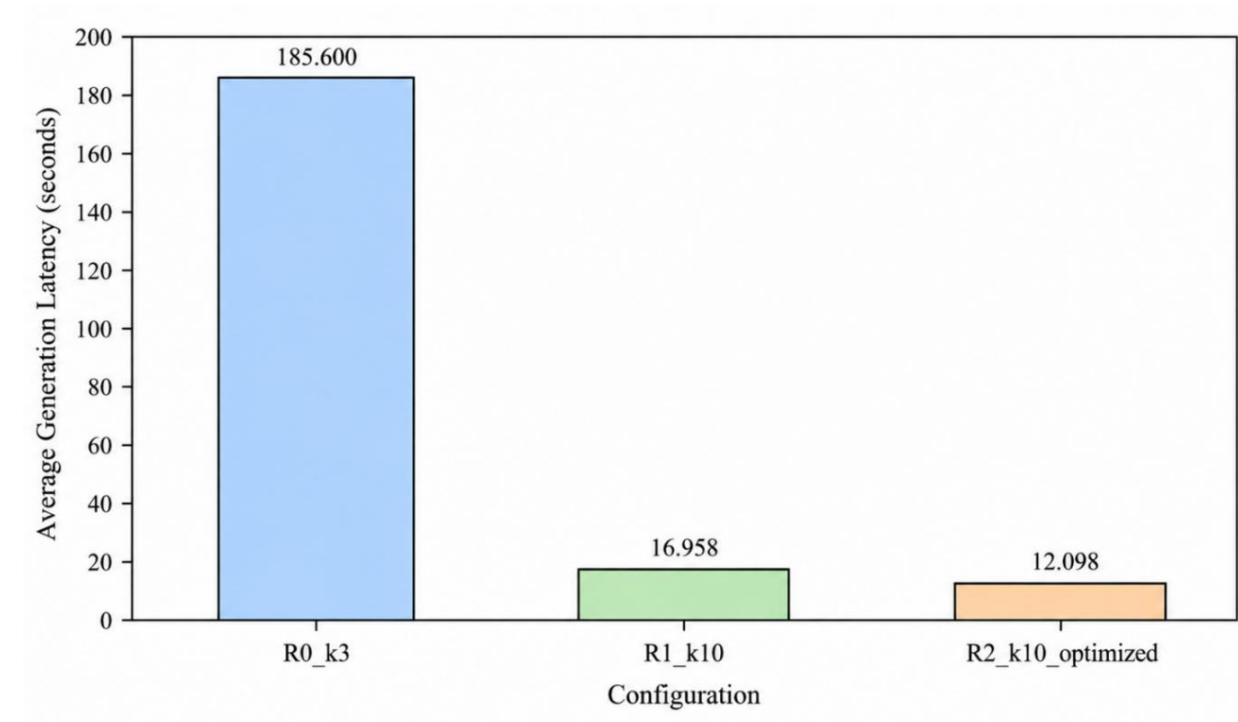


Figure 4.1.3: Average Generation Latency Across RAG Configurations

Table 4.1.2: Generation Performance of Different RAG Configurations

Metric	R0_k3	R1_k10	R2_k10_optimized
Questions	60	60	60
Successful	60	60	60
Errors	0	0	0
Mean Lexical F1	0.3030	0.3136	0.3046
IDK	23.33%	23.33%	25.00%
Mean Latency (s)	185.600	16.958	12.098
Mean Context Sections	2.45	3.23	3.10
Mean Context Characters	7014	8000	7862

Paired Lexical-F1 Difference

Table 4.1.3: Paired Lexical-F1 Differences and Wilcoxon Signed-Rank Test Results

Comparison	Mean $\Delta F1$	Wilcoxon p-value	Paired t-test p-value	Significant ($\alpha = 0.05$)
R0_k3 $\rightarrow$ R1_k10	+0.0106	0.263	0.133	No
R1_k10 $\rightarrow$ R2_k10_optimized	-0.0090	0.005	0.018	Yes
R0_k3 $\rightarrow$ R2_k10_optimized	+0.0016	0.942	0.801	No

A Wilcoxon signed-rank test was applied to the paired per-question lexical F1 scores ($n = 60$) for each of the three comparisons, since the same 60 questions were evaluated under every configuration. The comparison shows that increasing retrieval depth from 3 to 10 sections produced the largest positive change in mean lexical F1 (+0.0106), but this difference was not statistically significant (Wilcoxon $p = 0.263$). Context optimization subsequently produced a small mean reduction of -0.0090 relative to R1_k10, and this decrease was statistically significant (Wilcoxon $p = 0.005$; paired t-test $p = 0.018$). The overall difference between the

optimized configuration and the R0_k3 baseline was negligible and not significant (+0.0016, $p = 0.942$).

These results indicate that while R2_k10_optimized remained slightly above the R0_k3 baseline in raw mean F1, the reduction in F1 relative to R1_k10 was statistically significant according to the Wilcoxon signed-rank test ($p = 0.005$). In other words, the latency improvement gained through context optimization came with a small but statistically measurable cost in lexical answer overlap compared with using the full top-10 context.

4.2.2. Qualitative Analysis (Actual System Output)

The implemented RAG system accepts a document-based question, retrieves relevant document sections, and provides the selected context to the language model for answer generation. The final generation experiment used the Qwen2.5:3B model through Ollama. A sample question and generated answer are presented in Figure 4.1.4 to illustrate the operation of the implemented system.

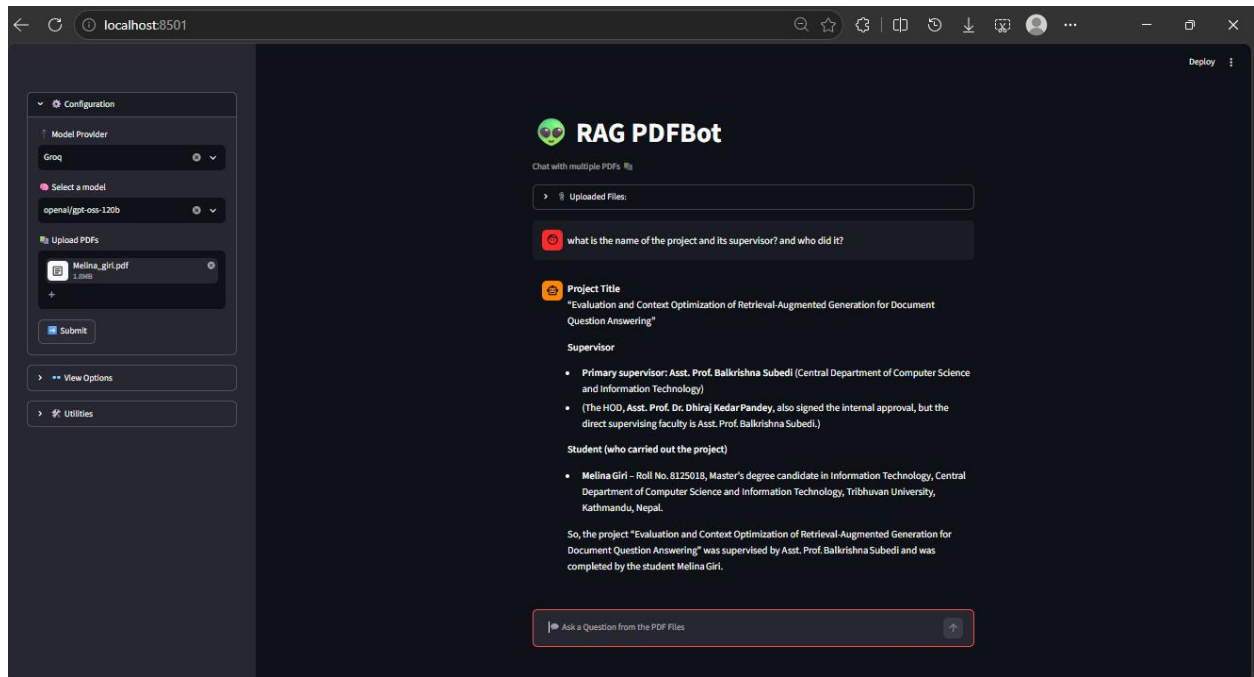


Figure 4.1.4: Sample Question and Generated Answer from the RAG System

The generated response demonstrates the complete process from question input and document retrieval to context provision and final answer generation.

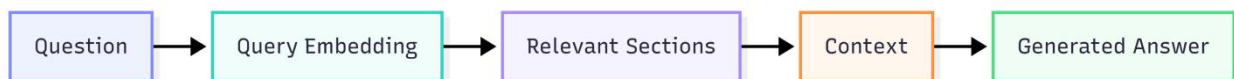


Figure 4.1.5: RAG Pipeline

The generated response is produced using the retrieved document context supplied to the language model as part of the RAG pipeline [1].

Analysis of the Three Configurations

R0_k3: The system uses only the top three retrieved sections. This provides a relatively small context but may omit information required for answering complex questions.

R1_k10: The system retrieves ten sections, providing more information to the language model. This configuration achieved the highest lexical F1 score (**0.3136**) among the three generation configurations.

R2_k10_optimized: The system initially retrieves ten sections and then removes redundant sections using Jaccard word-set overlap (≥ 0.70), retaining at most five sections. Although its lexical F1 (0.3046) was slightly lower than R1_k10, this difference was statistically significant (Wilcoxon $p = 0.005$). However, R2_k10_optimized achieved the lowest mean generation latency (12.098 seconds).

Overall, the experimental results show that increasing retrieval depth improved retrieval effectiveness, although the increase from R0_k3 to R1_k10 in mean lexical F1 was not statistically significant. Context optimization reduced generation latency but also produced a statistically significant reduction in lexical F1 relative to the unoptimized top-10 configuration. Since lexical F1 measures word overlap, these results should not be interpreted as a direct measure of semantic answer correctness.

CHAPTER 5: CONCLUSION

5.1. Conclusion

This research implemented and evaluated a Retrieval-Augmented Generation (RAG) system for document question answering using the Vectara Open RAG Benchmark. The implemented pipeline included document processing, embedding generation, vector-based retrieval, context selection, and language-model-based answer generation.

The retrieval evaluation showed that increasing retrieval depth from $k=3$ to $k=20$ improved retrieval effectiveness. Hit@ k increased from 61.18% at $k=3$ to 82.33% at $k=20$, while MRR increased from 0.5061 to 0.5385 and nDCG increased from 0.5333 to 0.6051.

The generation evaluation compared R0_k3, R1_k10, and R2_k10_optimized using 60 questions. R1_k10 achieved the highest mean lexical F1 of 0.3136. The difference between R0_k3 and R1_k10 was not statistically significant, whereas the reduction from R1_k10 to R2_k10_optimized was statistically significant (Wilcoxon $p = 0.005$).

Context optimization reduced mean generation latency from 16.958 seconds for R1_k10 to 12.098 seconds for R2_k10_optimized, while reducing mean lexical F1 from 0.3136 to 0.3046. Thus, within the evaluated configurations, context optimization reduced generation latency but involved a measurable reduction in lexical answer overlap. The results also show that lexical F1 should be interpreted as a word-overlap measure rather than a direct measure of semantic answer correctness.

5.2. Future Work

Based on the findings of this research, the following areas are recommended for future research and improvement:

1. Expand the generation evaluation dataset by evaluating a larger number of questions to improve the robustness of the results.
2. Improve context optimization by investigating semantic similarity, relevance, diversity, and learned ranking methods.
3. Evaluate different embedding models to examine their effects on retrieval performance.
4. Evaluate different language models to compare their effects on answer quality, latency, and resource requirements.
5. Use semantic answer evaluation methods such as BERTScore, embedding-based similarity, LLM-based evaluation, and human evaluation.
6. Improve retrieval and ranking methods to investigate whether more effective ranking can

improve retrieval quality.

7. Evaluate the RAG system on domain-specific document collections and real-world applications to assess its practical performance and generalizability.
8. Investigate runtime and generation latency variation across repeated executions and different system conditions.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Y. Wang, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [4] Vectara, “Open RAG Benchmark,” Vectara Research and Benchmark Dataset, 2024.
- [5] Qwen Team, “Qwen2.5: A Party of Foundation Models,” 2024.
- [6] Sentence-Transformers, “all-MiniLM-L12-v2,” Hugging Face Model Card, 2024.

Appendix

A.1 Main Interface Code

The code in Sections A.1–A.4 is drawn from the project's general-purpose interactive interface (supporting Groq/Gemini as configurable providers), separate from the fixed benchmark pipeline described in Section 2.4 that generated the reported results. The benchmark pipeline code is shown in Section A.5. This section shows the main interface of the RAG-based document question-answering system.

```
client > app.py x
1 import streamlit as st
2
3 from state.session import setup_session_state, is_chat_ready
4 from components.chat import (
5     render_chat_history,
6     render_download_chat_history,
7     render_uploaded_files_expander,
8     render_user_input
9 )
10 from components.sidebar import (
11     render_model_selector,
12     render_view_selector,
13     sidebar_file_upload,
14     sidebar_provider_change_check,
15     sidebar_utilities
16 )
17 from components.inspector import render_inspect_query
18
19
20 def main():
21     st.set_page_config(page_title="RAG PDFBot", layout="centered")
22     st.title("🤖 RAG PDFBot")
23     st.caption("Chat with multiple PDFs :books:")
24
25     setup_session_state()
26
27     if st.session_state.get("chat_history"):
28         render_download_chat_history()
29
30     with st.sidebar:
31         with st.expander("⚙️ Configuration", expanded=True):
32             model_provider, model = render_model_selector()
33             sidebar_file_upload(model_provider)
34             sidebar_provider_change_check(model_provider, model)
35
36         view_option = render_view_selector()
37         sidebar_utilities()
38
39     if not st.session_state.get(f"uploaded_files_{st.session_state.uploader_key}", []):
40         st.info("📄 Please upload and submit PDFs to start chatting.")
41
42     if st.session_state.get("unsubmitted_files", False):
43         st.warning("📄 New PDFs uploaded. Please submit before chatting.")
44
45     if st.session_state.get("chat_ready") and st.session_state.get("pdf_files", []):
46         render_uploaded_files_expander()
47
48     if view_option == "💬 Chat":
49         if st.session_state.get("chat_history", []):
50             render_chat_history()
51
52         if is_chat_ready():
53             render_user_input(model_provider, model)
54     elif view_option == "🔍 Inspector":
55         if is_chat_ready():
56             render_inspect_query(model_provider)
57
58 if __name__ == "__main__":
59     main()
```

Figure A.1: Main Interface Code

A.2 Loading Documents and Corpus Sections

This section demonstrates how the benchmark corpus is loaded and its pre-segmented document sections are prepared for embedding generation and retrieval. The document-processing pipeline includes:

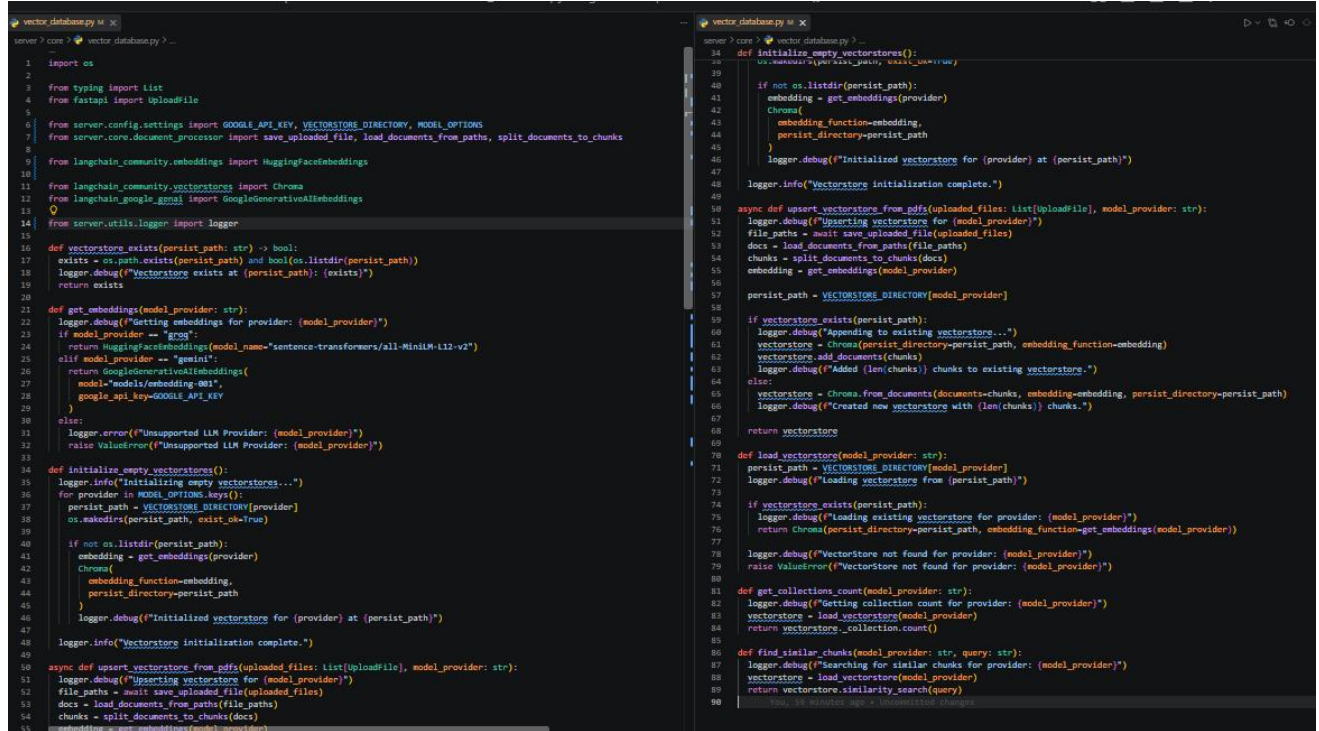
- Loading the benchmark corpus JSON files
- Reading each paper’s pre-segmented sections
- Preparing document and section metadata
- Generating embeddings

```
document_processor.py 14 X
server > code > document_processor.py > ...
You, 54 minutes ago | 2 authors (dash05 and one other)
1 import os
2 import aiofiles
3
4 from typing import List
5 from fastapi import UploadFile
6
7 from langchain_community.document_loaders import PyPDFLoader
8 from langchain_text_splitters import TokenTextSplitter
9
10 from server.config.settings import TEMPFILE_UPLOAD_DIRECTORY
11 from server.utils.logger import logger
12
13 def validate_pdf(file: UploadFile, max_size_mb: int = 200):
14     if not file.filename.endswith(".pdf"):
15         logger.warning(f"Invalid file type: {file.filename}")
16         raise ValueError(f"{file.filename} is not a valid PDF file.")
17
18     file_size_mb = len(file.file.read()) / (1024 * 1024)
19     file.file.seek(0)
20
21     if file_size_mb > max_size_mb:
22         logger.warning(f"File too large: {file.filename} ({file_size_mb:.2f} MB)")
23         raise ValueError(f"PDF file size exceeds the maximum allowed size of {max_size_mb} MB.")
24
25     logger.debug(f"Validated PDF: {file.filename} ({file_size_mb:.2f} MB)")
26
27 async def save_uploaded_file(files: List[UploadFile]) -> List[str]:
28     os.makedirs(TEMPFILE_UPLOAD_DIRECTORY, exist_ok=True)
29     file_paths = []
30
31     for file in files:
32         validate_pdf(file)
33         file_path = os.path.join(TEMPFILE_UPLOAD_DIRECTORY, file.filename)
34         async with aiofiles.open(file_path, "wb") as f:
35             content = await file.read()
36             await f.write(content)
37         file_paths.append(file_path)
38         logger.debug(f"Saved uploaded file: {file.filename} to {file_path}")
39
40     return file_paths
41
42 def load_documents_from_paths(file_paths: List[str]):
43     docs = []
44     for file_path in file_paths:
45         loader = PyPDFLoader(file_path)
46         loaded = loader.load()
47         logger.debug(f"Loaded {len(loaded)} documents from {file_path}")
48         docs.extend(loaded)
49
50     return docs
51
52 def split_documents_to_chunks(docs) -> List[str]:
53     text_splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)
54     chunks = text_splitter.split_documents(docs)
55     logger.debug(f"Split {len(docs)} docs into {len(chunks)} chunks")
56     return chunks
57
You, 54 minutes ago • Uncommitted changes
```

Figure A.2: Corpus Loading and Section Preparation Code

A.3 Embedding Generation and Vector Retrieval

The following code implements the generation of dense vector representations using the all-MiniLM-L12-v2 embedding model [6]. The generated vectors are used to calculate similarity between the query and document sections.



```
vector_database.py
1 import os
2
3 from typing import List
4 from fastapi import UploadFile
5
6 from server.config.settings import GOOGLE_API_KEY, VECTORSTORE_DIRECTORY, MODEL_OPTIONS
7 from server.core.document_processor import save_uploaded_file, load_documents_from_paths, split_documents_to_chunks
8
9 from langchain_community.embeddings import HuggingFaceEmbeddings
10
11 from langchain_community.vectorstores import Chroma
12 from langchain_google_genai import GoogleGenerativeAIEmbeddings
13
14 from server.utils.logger import logger
15
16 def vectorstore_exists(persist_path: str) -> bool:
17     exists = os.path.exists(persist_path) and bool(os.listdir(persist_path))
18     logger.debug(f"Vectorstore exists at {persist_path}: {exists}")
19     return exists
20
21 def get_embeddings(model_provider: str):
22     logger.debug(f"Getting embeddings for provider: {model_provider}")
23     if model_provider == "gpt4":
24         return HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L12-v2")
25     elif model_provider == "gemini":
26         return GoogleGenerativeAIEmbeddings(
27             model="models/embedding-001",
28             google_api_key=GOOGLE_API_KEY
29         )
30     else:
31         logger.error(f"Unsupported LLM Provider: {model_provider}")
32         raise ValueError(f"Unsupported LLM Provider: {model_provider}")
33
34 def initialize_empty_vectorstores():
35     logger.info("Initializing empty vectorstores...")
36     for provider in MODEL_OPTIONS.keys():
37         persist_path = VECTORSTORE_DIRECTORY[provider]
38         os.makedirs(persist_path, exist_ok=True)
39
40         if not os.listdir(persist_path):
41             embedding = get_embeddings(provider)
42             Chroma(
43                 embedding_function=embedding,
44                 persist_directory=persist_path
45             )
46             logger.debug(f"Initialized vectorstore for {provider} at {persist_path}")
47
48     logger.info("Vectorstore initialization complete.")
49
50 async def upsert_vectorstore_from_pdf(uploaded_files: List[UploadFile], model_provider: str):
51     logger.debug(f"Upserting vectorstore for {model_provider}")
52     file_paths = await save_uploaded_files(uploaded_files)
53     docs = load_documents_from_paths(file_paths)
54     chunks = split_documents_to_chunks(docs)
55     embedding = get_embeddings(model_provider)
56
57     persist_path = VECTORSTORE_DIRECTORY[model_provider]
58
59     if vectorstore_exists(persist_path):
60         logger.debug("Appending to existing vectorstore...")
61         vectorstore = Chroma(persist_directory=persist_path, embedding_function=embedding)
62         vectorstore.add_documents(chunks)
63         logger.debug(f"Added {len(chunks)} chunks to existing vectorstore.")
64     else:
65         vectorstore = Chroma.from_documents(docs, chunks, embedding=embedding, persist_directory=persist_path)
66         logger.debug(f"Created new vectorstore with {len(chunks)} chunks.")
67
68     return vectorstore
69
70 def load_vectorstore(model_provider: str):
71     persist_path = VECTORSTORE_DIRECTORY[model_provider]
72     logger.debug(f"Loading vectorstore from {persist_path}")
73
74     if vectorstore_exists(persist_path):
75         logger.debug(f"Loading existing vectorstore for provider: {model_provider}")
76         return Chroma(persist_directory=persist_path, embedding_function=get_embeddings(model_provider))
77
78     logger.debug(f"Vectorstore not found for provider: {model_provider}")
79     raise ValueError(f"Vectorstore not found for provider: {model_provider}")
80
81 def get_collections_count(model_provider: str):
82     logger.debug(f"Getting collection count for provider: {model_provider}")
83     vectorstore = load_vectorstore(model_provider)
84     return vectorstore.collection.count()
85
86 def find_similar_chunks(model_provider: str, query: str):
87     logger.debug(f"Searching for similar chunks for provider: {model_provider}")
88     vectorstore = load_vectorstore(model_provider)
89     return vectorstore.similarity_search(query)
90
```

```
vector_store.py
1 def initialize_empty_vectorstores():
2     os.makedirs(persist_path, exist_ok=True)
3
4     if not os.listdir(persist_path):
5         embedding = get_embeddings(provider)
6         Chroma(
7             embedding_function=embedding,
8             persist_directory=persist_path
9         )
10        logger.debug(f"Initialized vectorstore for {provider} at {persist_path}")
11
12    logger.info("Vectorstore initialization complete.")
13
14    async def upsert_vectorstore_from_pdf(uploaded_files: List[UploadFile], model_provider: str):
15        logger.debug(f"Upserting vectorstore for {model_provider}")
16        file_paths = await save_uploaded_files(uploaded_files)
17        docs = load_documents_from_paths(file_paths)
18        chunks = split_documents_to_chunks(docs)
19        embedding = get_embeddings(model_provider)
20
21        persist_path = VECTORSTORE_DIRECTORY[model_provider]
22
23        if vectorstore_exists(persist_path):
24            logger.debug("Appending to existing vectorstore...")
25            vectorstore = Chroma(persist_directory=persist_path, embedding_function=embedding)
26            vectorstore.add_documents(chunks)
27            logger.debug(f"Added {len(chunks)} chunks to existing vectorstore.")
28        else:
29            vectorstore = Chroma.from_documents(docs, chunks, embedding=embedding, persist_directory=persist_path)
30            logger.debug(f"Created new vectorstore with {len(chunks)} chunks.")
31
32        return vectorstore
33
34    def load_vectorstore(model_provider: str):
35        persist_path = VECTORSTORE_DIRECTORY[model_provider]
36        logger.debug(f"Loading vectorstore from {persist_path}")
37
38        if vectorstore_exists(persist_path):
39            logger.debug(f"Loading existing vectorstore for provider: {model_provider}")
40            return Chroma(persist_directory=persist_path, embedding_function=get_embeddings(model_provider))
41
42        logger.debug(f"Vectorstore not found for provider: {model_provider}")
43        raise ValueError(f"Vectorstore not found for provider: {model_provider}")
44
45    def get_collections_count(model_provider: str):
46        logger.debug(f"Getting collection count for provider: {model_provider}")
47        vectorstore = load_vectorstore(model_provider)
48        return vectorstore.collection.count()
49
50    def find_similar_chunks(model_provider: str, query: str):
51        logger.debug(f"Searching for similar chunks for provider: {model_provider}")
52        vectorstore = load_vectorstore(model_provider)
53        return vectorstore.similarity_search(query)
54
```

Figure A.3: Embedding Generation and Similarity-Based Retrieval Code

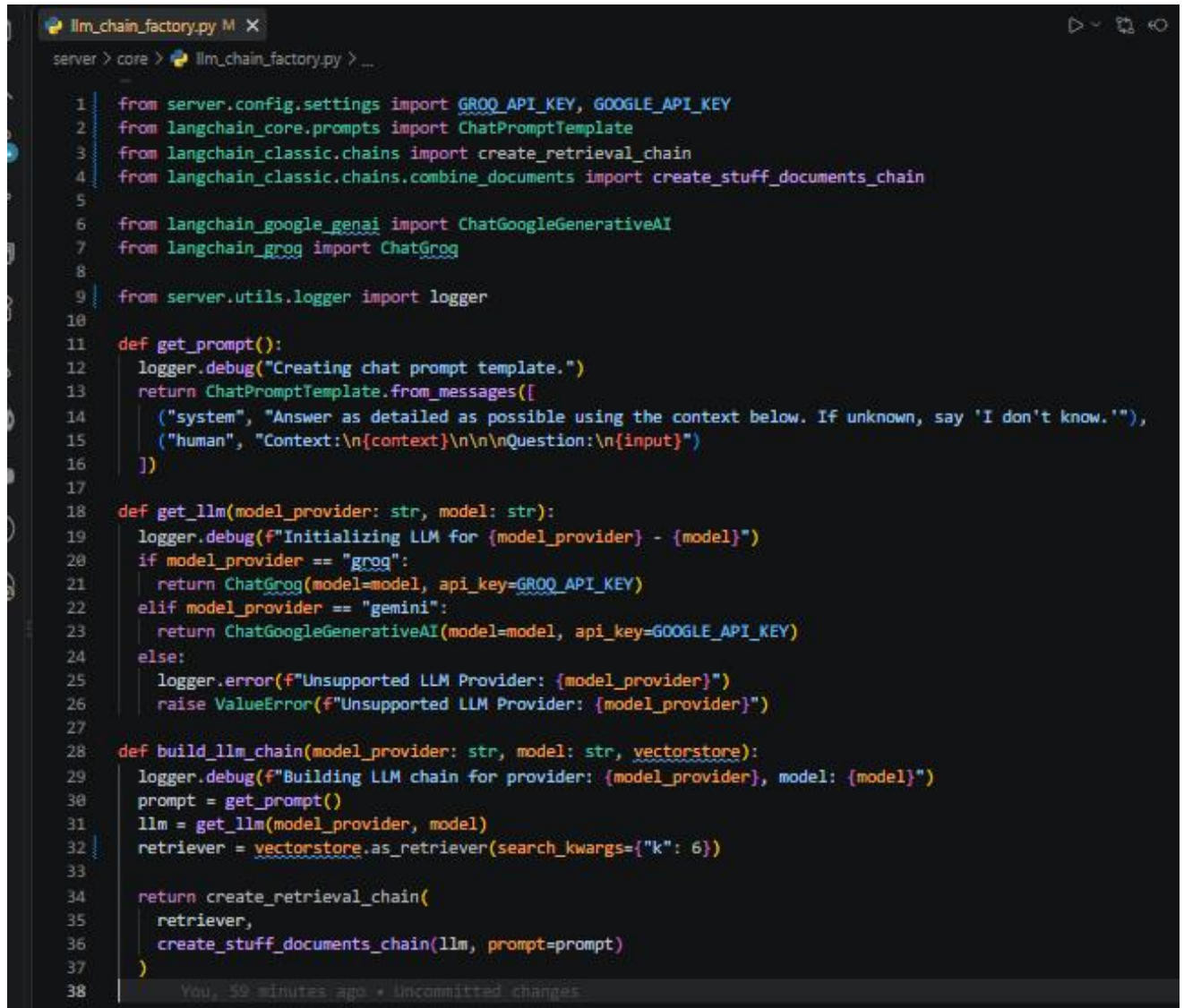
A.4 RAG Answer Generation Code

This code demonstrates how the retrieved document context is combined with the user's question and passed to the language model.

The system follows the sequence:

User Question → Retrieved Sections → (Optional) Context Optimization → Character-Limited Context → Prompt → Language Model → Generated Answer

The final generation experiment used the locally executed Qwen2.5:3B model through Ollama.



```
llm_chain_factory.py M X
server > core > llm_chain_factory.py > ...

1  from server.config.settings import GROQ_API_KEY, GOOGLE_API_KEY
2  from langchain_core.prompts import ChatPromptTemplate
3  from langchain_classic.chains import create_retrieval_chain
4  from langchain_classic.chains.combine_documents import create_stuff_documents_chain
5
6  from langchain_google_genai import ChatGoogleGenerativeAI
7  from langchain_groq import ChatGroq
8
9  from server.utils.logger import logger
10
11 def get_prompt():
12     logger.debug("Creating chat prompt template.")
13     return ChatPromptTemplate.from_messages([
14         ("system", "Answer as detailed as possible using the context below. If unknown, say 'I don't know.'"),
15         ("human", "Context:\n{context}\n\nQuestion:\n{input}")
16     ])
17
18 def get_llm(model_provider: str, model: str):
19     logger.debug(f"Initializing LLM for {model_provider} - {model}")
20     if model_provider == "groq":
21         return ChatGroq(model=model, api_key=GROQ_API_KEY)
22     elif model_provider == "gemini":
23         return ChatGoogleGenerativeAI(model=model, api_key=GOOGLE_API_KEY)
24     else:
25         logger.error(f"Unsupported LLM Provider: {model_provider}")
26         raise ValueError(f"Unsupported LLM Provider: {model_provider}")
27
28 def build_llm_chain(model_provider: str, model: str, vectorstore):
29     logger.debug(f"Building LLM chain for provider: {model_provider}, model: {model}")
30     prompt = get_prompt()
31     llm = get_llm(model_provider, model)
32     retriever = vectorstore.as_retriever(search_kwargs={"k": 6})
33
34     return create_retrieval_chain(
35         retriever,
36         create_stuff_documents_chain(llm, prompt=prompt)
37     )
38
You, 59 minutes ago • Uncommitted changes
```

Figure A.4: RAG Answer Generation Code

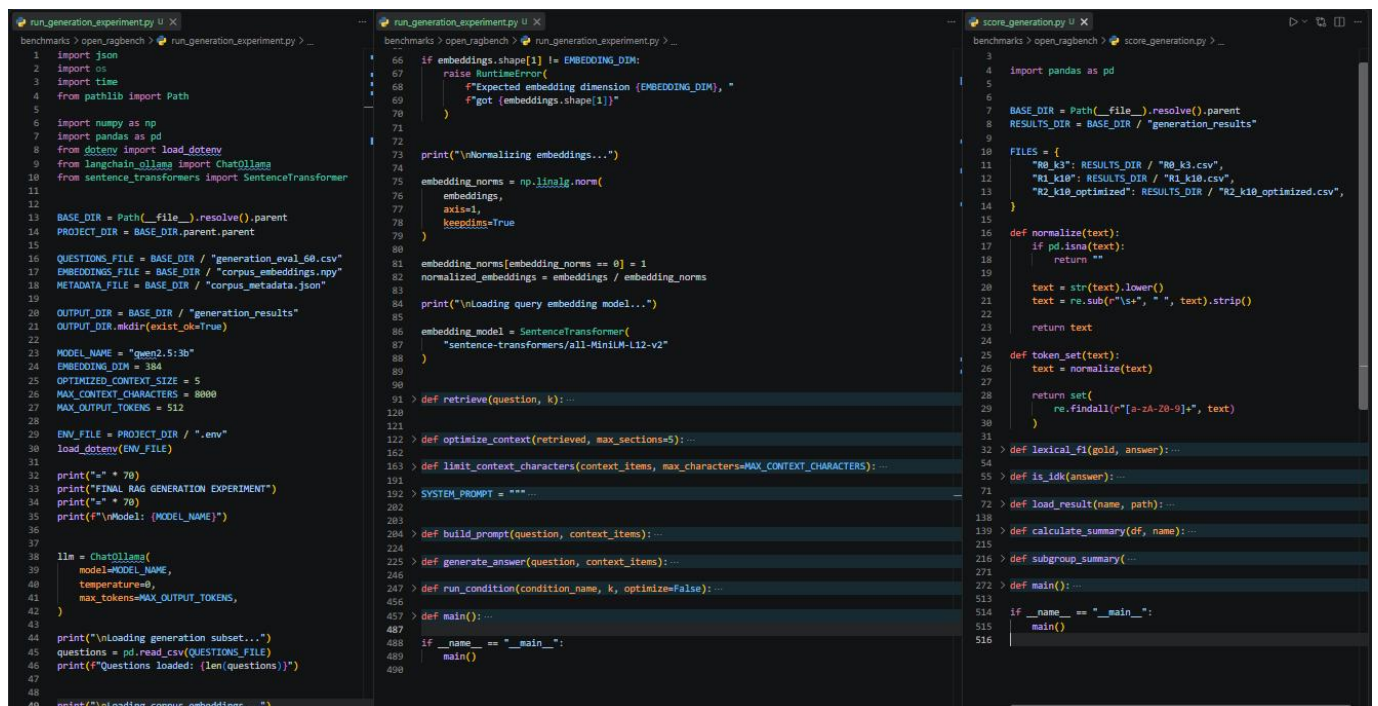
A.5 Evaluation and Result Analysis Code

This section contains the code used to evaluate the RAG system. Retrieval performance is evaluated using Hit@k, MRR, and nDCG, while generation performance is evaluated using lexical F1, IDK percentage, latency, and context size.

The generation experiment compares:

- R0_k3
- R1_k10
- R2_k10_optimized

The resulting evaluation tables are generated for overall performance, question type, source type, and paired lexical-F1 differences.



```
run_generation_experiment.py
1 import json
2 import os
3 import time
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8 from dotenv import load_dotenv
9 from langchain_ollama import ChatOllama
10 from sentence_transformers import SentenceTransformer
11
12 BASE_DIR = Path(__file__).resolve().parent
13 PROJECT_DIR = BASE_DIR.parent.parent
14
15 QUESTIONS_FILE = BASE_DIR / "generation_eval_68.csv"
16 EMBEDDINGS_FILE = BASE_DIR / "corpus_embeddings.npy"
17 METADATA_FILE = BASE_DIR / "corpus_metadata.json"
18
19 OUTPUT_DIR = BASE_DIR / "generation_results"
20 OUTPUT_DIR.mkdir(exist_ok=True)
21
22 MODEL_NAME = "qwen2.5:3b"
23 EMBEDDING_DIM = 384
24 OPTIMIZED_CONTEXT_SIZE = 5
25 MAX_CONTEXT_CHARACTERS = 8000
26 MAX_OUTPUT_TOKENS = 512
27
28 ENV_FILE = PROJECT_DIR / ".env"
29 load_dotenv(ENV_FILE)
30
31 print(f"{'=' * 70}")
32 print("FINAL RAG GENERATION EXPERIMENT")
33 print(f"{'=' * 70}")
34 print(f"Model: {MODEL_NAME}")
35
36 llm = ChatOllama(
37     model=MODEL_NAME,
38     temperature=0,
39     max_tokens=MAX_OUTPUT_TOKENS,
40 )
41
42 print("Loading generation subset...")
43 questions = pd.read_csv(QUESTIONS_FILE)
44 print(f"Questions loaded: {len(questions)}")
45
46 print("Loading corpus embeddings...")
47
48 run_generation_experiment.py
49
50 if embeddings.shape[1] != EMBEDDING_DIM:
51     raise RuntimeError(
52         f"Expected embedding dimension {EMBEDDING_DIM}, "
53         f"got {embeddings.shape[1]}"
54     )
55
56 print("Normalizing embeddings...")
57
58 embedding_norms = np.linalg.norm(
59     embeddings,
60     axis=1,
61     keepdims=True
62 )
63
64 embedding_norms[embedding_norms == 0] = 1
65 normalized_embeddings = embeddings / embedding_norms
66
67 print("Loading query embedding model...")
68
69 embedding_model = SentenceTransformer(
70     "sentence-transformers/all-MiniLM-L12-v2"
71 )
72
73 def retrieve(question, k):
74     ...
75
76 def optimize_context(retrieved, max_sections=5):
77     ...
78
79 def limit_context_characters(context_items, max_characters=MAX_CONTEXT_CHARACTERS):
80     ...
81
82 SYSTEM_PROMPT = """
83 ...
84 ...
85 ...
86 ...
87 ...
88 ...
89 ...
90 ...
91 ...
92 ...
93 ...
94 ...
95 ...
96 ...
97 ...
98 ...
99 ...
100 ...
101 ...
102 ...
103 ...
104 ...
105 ...
106 ...
107 ...
108 ...
109 ...
110 ...
111 ...
112 ...
113 ...
114 ...
115 ...
116 ...
117 ...
118 ...
119 ...
120 ...
121 ...
122 ...
123 ...
124 ...
125 ...
126 ...
127 ...
128 ...
129 ...
130 ...
131 ...
132 ...
133 ...
134 ...
135 ...
136 ...
137 ...
138 ...
139 ...
140 ...
141 ...
142 ...
143 ...
144 ...
145 ...
146 ...
147 ...
148 ...
149 ...
150 ...
151 ...
152 ...
153 ...
154 ...
155 ...
156 ...
157 ...
158 ...
159 ...
160 ...
161 ...
162 ...
163 ...
164 ...
165 ...
166 ...
167 ...
168 ...
169 ...
170 ...
171 ...
172 ...
173 ...
174 ...
175 ...
176 ...
177 ...
178 ...
179 ...
180 ...
181 ...
182 ...
183 ...
184 ...
185 ...
186 ...
187 ...
188 ...
189 ...
190 ...
191 ...
192 ...
193 ...
194 ...
195 ...
196 ...
197 ...
198 ...
199 ...
200 ...
201 ...
202 ...
203 ...
204 ...
205 ...
206 ...
207 ...
208 ...
209 ...
210 ...
211 ...
212 ...
213 ...
214 ...
215 ...
216 ...
217 ...
218 ...
219 ...
220 ...
221 ...
222 ...
223 ...
224 ...
225 ...
226 ...
227 ...
228 ...
229 ...
230 ...
231 ...
232 ...
233 ...
234 ...
235 ...
236 ...
237 ...
238 ...
239 ...
240 ...
241 ...
242 ...
243 ...
244 ...
245 ...
246 ...
247 ...
248 ...
249 ...
250 ...
251 ...
252 ...
253 ...
254 ...
255 ...
256 ...
257 ...
258 ...
259 ...
260 ...
261 ...
262 ...
263 ...
264 ...
265 ...
266 ...
267 ...
268 ...
269 ...
270 ...
271 ...
272 ...
273 ...
274 ...
275 ...
276 ...
277 ...
278 ...
279 ...
280 ...
281 ...
282 ...
283 ...
284 ...
285 ...
286 ...
287 ...
288 ...
289 ...
290 ...
291 ...
292 ...
293 ...
294 ...
295 ...
296 ...
297 ...
298 ...
299 ...
300 ...
301 ...
302 ...
303 ...
304 ...
305 ...
306 ...
307 ...
308 ...
309 ...
310 ...
311 ...
312 ...
313 ...
314 ...
315 ...
316 ...
317 ...
318 ...
319 ...
320 ...
321 ...
322 ...
323 ...
324 ...
325 ...
326 ...
327 ...
328 ...
329 ...
330 ...
331 ...
332 ...
333 ...
334 ...
335 ...
336 ...
337 ...
338 ...
339 ...
340 ...
341 ...
342 ...
343 ...
344 ...
345 ...
346 ...
347 ...
348 ...
349 ...
350 ...
351 ...
352 ...
353 ...
354 ...
355 ...
356 ...
357 ...
358 ...
359 ...
360 ...
361 ...
362 ...
363 ...
364 ...
365 ...
366 ...
367 ...
368 ...
369 ...
370 ...
371 ...
372 ...
373 ...
374 ...
375 ...
376 ...
377 ...
378 ...
379 ...
380 ...
381 ...
382 ...
383 ...
384 ...
385 ...
386 ...
387 ...
388 ...
389 ...
390 ...
391 ...
392 ...
393 ...
394 ...
395 ...
396 ...
397 ...
398 ...
399 ...
400 ...
401 ...
402 ...
403 ...
404 ...
405 ...
406 ...
407 ...
408 ...
409 ...
410 ...
411 ...
412 ...
413 ...
414 ...
415 ...
416 ...
417 ...
418 ...
419 ...
420 ...
421 ...
422 ...
423 ...
424 ...
425 ...
426 ...
427 ...
428 ...
429 ...
430 ...
431 ...
432 ...
433 ...
434 ...
435 ...
436 ...
437 ...
438 ...
439 ...
440 ...
441 ...
442 ...
443 ...
444 ...
445 ...
446 ...
447 ...
448 ...
449 ...
450 ...
451 ...
452 ...
453 ...
454 ...
455 ...
456 ...
457 ...
458 ...
459 ...
460 ...
461 ...
462 ...
463 ...
464 ...
465 ...
466 ...
467 ...
468 ...
469 ...
470 ...
471 ...
472 ...
473 ...
474 ...
475 ...
476 ...
477 ...
478 ...
479 ...
480 ...
481 ...
482 ...
483 ...
484 ...
485 ...
486 ...
487 ...
488 ...
489 ...
490 ...
491 ...
492 ...
493 ...
494 ...
495 ...
496 ...
497 ...
498 ...
499 ...
500 ...
501 ...
502 ...
503 ...
504 ...
505 ...
506 ...
507 ...
508 ...
509 ...
510 ...
511 ...
512 ...
513 ...
514 ...
515 ...
516 ...
517 ...
518 ...
519 ...
520 ...
521 ...
522 ...
523 ...
524 ...
525 ...
526 ...
527 ...
528 ...
529 ...
530 ...
531 ...
532 ...
533 ...
534 ...
535 ...
536 ...
537 ...
538 ...
539 ...
540 ...
541 ...
542 ...
543 ...
544 ...
545 ...
546 ...
547 ...
548 ...
549 ...
550 ...
551 ...
552 ...
553 ...
554 ...
555 ...
556 ...
557 ...
558 ...
559 ...
560 ...
561 ...
562 ...
563 ...
564 ...
565 ...
566 ...
567 ...
568 ...
569 ...
570 ...
571 ...
572 ...
573 ...
574 ...
575 ...
576 ...
577 ...
578 ...
579 ...
580 ...
581 ...
582 ...
583 ...
584 ...
585 ...
586 ...
587 ...
588 ...
589 ...
590 ...
591 ...
592 ...
593 ...
594 ...
595 ...
596 ...
597 ...
598 ...
599 ...
600 ...
601 ...
602 ...
603 ...
604 ...
605 ...
606 ...
607 ...
608 ...
609 ...
610 ...
611 ...
612 ...
613 ...
614 ...
615 ...
616 ...
617 ...
618 ...
619 ...
620 ...
621 ...
622 ...
623 ...
624 ...
625 ...
626 ...
627 ...
628 ...
629 ...
630 ...
631 ...
632 ...
633 ...
634 ...
635 ...
636 ...
637 ...
638 ...
639 ...
640 ...
641 ...
642 ...
643 ...
644 ...
645 ...
646 ...
647 ...
648 ...
649 ...
650 ...
651 ...
652 ...
653 ...
654 ...
655 ...
656 ...
657 ...
658 ...
659 ...
660 ...
661 ...
662 ...
663 ...
664 ...
665 ...
666 ...
667 ...
668 ...
669 ...
670 ...
671 ...
672 ...
673 ...
674 ...
675 ...
676 ...
677 ...
678 ...
679 ...
680 ...
681 ...
682 ...
683 ...
684 ...
685 ...
686 ...
687 ...
688 ...
689 ...
690 ...
691 ...
692 ...
693 ...
694 ...
695 ...
696 ...
697 ...
698 ...
699 ...
700 ...
701 ...
702 ...
703 ...
704 ...
705 ...
706 ...
707 ...
708 ...
709 ...
710 ...
711 ...
712 ...
713 ...
714 ...
715 ...
716 ...
717 ...
718 ...
719 ...
720 ...
721 ...
722 ...
723 ...
724 ...
725 ...
726 ...
727 ...
728 ...
729 ...
730 ...
731 ...
732 ...
733 ...
734 ...
735 ...
736 ...
737 ...
738 ...
739 ...
740 ...
741 ...
742 ...
743 ...
744 ...
745 ...
746 ...
747 ...
748 ...
749 ...
750 ...
751 ...
752 ...
753 ...
754 ...
755 ...
756 ...
757 ...
758 ...
759 ...
760 ...
761 ...
762 ...
763 ...
764 ...
765 ...
766 ...
767 ...
768 ...
769 ...
770 ...
771 ...
772 ...
773 ...
774 ...
775 ...
776 ...
777 ...
778 ...
779 ...
780 ...
781 ...
782 ...
783 ...
784 ...
785 ...
786 ...
787 ...
788 ...
789 ...
790 ...
791 ...
792 ...
793 ...
794 ...
795 ...
796 ...
797 ...
798 ...
799 ...
800 ...
801 ...
802 ...
803 ...
804 ...
805 ...
806 ...
807 ...
808 ...
809 ...
810 ...
811 ...
812 ...
813 ...
814 ...
815 ...
816 ...
817 ...
818 ...
819 ...
820 ...
821 ...
822 ...
823 ...
824 ...
825 ...
826 ...
827 ...
828 ...
829 ...
830 ...
831 ...
832 ...
833 ...
834 ...
835 ...
836 ...
837 ...
838 ...
839 ...
840 ...
841 ...
842 ...
843 ...
844 ...
845 ...
846 ...
847 ...
848 ...
849 ...
850 ...
851 ...
852 ...
853 ...
854 ...
855 ...
856 ...
857 ...
858 ...
859 ...
860 ...
861 ...
862 ...
863 ...
864 ...
865 ...
866 ...
867 ...
868 ...
869 ...
870 ...
871 ...
872 ...
873 ...
874 ...
875 ...
876 ...
877 ...
878 ...
879 ...
880 ...
881 ...
882 ...
883 ...
884 ...
885 ...
886 ...
887 ...
888 ...
889 ...
890 ...
891 ...
892 ...
893 ...
894 ...
895 ...
896 ...
897 ...
898 ...
899 ...
900 ...
901 ...
902 ...
903 ...
904 ...
905 ...
906 ...
907 ...
908 ...
909 ...
910 ...
911 ...
912 ...
913 ...
914 ...
915 ...
916 ...
917 ...
918 ...
919 ...
920 ...
921 ...
922 ...
923 ...
924 ...
925 ...
926 ...
927 ...
928 ...
929 ...
930 ...
931 ...
932 ...
933 ...
934 ...
935 ...
936 ...
937 ...
938 ...
939 ...
940 ...
941 ...
942 ...
943 ...
944 ...
945 ...
946 ...
947 ...
948 ...
949 ...
950 ...
951 ...
952 ...
953 ...
954 ...
955 ...
956 ...
957 ...
958 ...
959 ...
960 ...
961 ...
962 ...
963 ...
964 ...
965 ...
966 ...
967 ...
968 ...
969 ...
970 ...
971 ...
972 ...
973 ...
974 ...
975 ...
976 ...
977 ...
978 ...
979 ...
980 ...
981 ...
982 ...
983 ...
984 ...
985 ...
986 ...
987 ...
988 ...
989 ...
990 ...
991 ...
992 ...
993 ...
994 ...
995 ...
996 ...
997 ...
998 ...
999 ...
1000 ...
1001 ...
1002 ...
1003 ...
1004 ...
1005 ...
1006 ...
1007 ...
1008 ...
1009 ...
1010 ...
1011 ...
1012 ...
1013 ...
1014 ...
1015 ...
1016 ...
1017 ...
1018 ...
1019 ...
1020 ...
1021 ...
1022 ...
1023 ...
1024 ...
1025 ...
1026 ...
1027 ...
1028 ...
1029 ...
1030 ...
1031 ...
1032 ...
1033 ...
1034 ...
1035 ...
1036 ...
1037 ...
1038 ...
1039 ...
1040 ...
1041 ...
1042 ...
1043 ...
1044 ...
1045 ...
1046 ...
1047 ...
1048 ...
1049 ...
1050 ...
1051 ...
1052 ...
1053 ...
1054 ...
1055 ...
1056 ...
1057 ...
1058 ...
1059 ...
1060 ...
1061 ...
1062 ...
1063 ...
1064 ...
1065 ...
1066 ...
1067 ...
1068 ...
1069 ...
1070 ...
1071 ...
1072 ...
1073 ...
1074 ...
1075 ...
1076 ...
1077 ...
1078 ...
1079 ...
1080 ...
1081 ...
1082 ...
1083 ...
1084 ...
1085 ...
1086 ...
1087 ...
1088 ...
1089 ...
1090 ...
1091 ...
1092 ...
1093 ...
1094 ...
1095 ...
1096 ...
1097 ...
1098 ...
1099 ...
1100 ...
1101 ...
1102 ...
1103 ...
1104 ...
1105 ...
1106 ...
1107 ...
1108 ...
1109 ...
1110 ...
1111 ...
1112 ...
1113 ...
1114 ...
1115 ...
1116 ...
1117 ...
1118 ...
1119 ...
1120 ...
1121 ...
1122 ...
1123 ...
1124 ...
1125 ...
1126 ...
1127 ...
1128 ...
1129 ...
1130 ...
1131 ...
1132 ...
1133 ...
1134 ...
1135 ...
1136 ...
1137 ...
1138 ...
1139 ...
1140 ...
1141 ...
1142 ...
1143 ...
1144 ...
1145 ...
1146 ...
1147 ...
1148 ...
1149 ...
1150 ...
1151 ...
1152 ...
1153 ...
1154 ...
1155 ...
1156 ...
1157 ...
1158 ...
1159 ...
1160 ...
1161 ...
1162 ...
1163 ...
1164 ...
1165 ...
1166 ...
1167 ...
1168 ...
1169 ...
1170 ...
1171 ...
1172 ...
1173 ...
1174 ...
1175 ...
1176 ...
1177 ...
1178 ...
1179 ...
1180 ...
1181 ...
1182 ...
1183 ...
1184 ...
1185 ...
1186 ...
1187 ...
1188 ...
1189 ...
1190 ...
1191 ...
1192 ...
1193 ...
1194 ...
1195 ...
1196 ...
1197 ...
1198 ...
1199 ...
1200 ...
1201 ...
1202 ...
1203 ...
1204 ...
1205 ...
1206 ...
1207 ...
1208 ...
1209 ...
1210 ...
1211 ...
1212 ...
1213 ...
1214 ...
1215 ...
1216 ...
1217 ...
1218 ...
1219 ...
1220 ...
1221 ...
1222 ...
1223 ...
1224 ...
1225 ...
1226 ...
1227 ...
1228 ...
1229 ...
1230 ...
1231 ...
1232 ...
1233 ...
1234 ...
1235 ...
1236 ...
1237 ...
1238 ...
1239 ...
1240 ...
1241 ...
1242 ...
1243 ...
1244 ...
1245 ...
1246 ...
1247 ...
1248 ...
1249 ...
1250 ...
1251 ...
1252 ...
1253 ...
1254 ...
1255 ...
1256 ...
1257 ...
1258 ...
1259 ...
1260 ...
1261 ...
1262 ...
1263 ...
1264 ...
1265 ...
1266 ...
1267 ...
1268 ...
1269 ...
1270 ...
1271 ...
1272 ...
1273 ...
1274 ...
1275 ...
1276 ...
1277 ...
1278 ...
1279 ...
1280 ...
1281 ...
1282 ...
1283 ...
1284 ...
1285 ...
1286 ...
1287 ...
1288 ...
1289 ...
1290 ...
1291 ...
1292 ...
1293 ...
1294 ...
1295 ...
1296 ...
1297 ...
1298 ...
1299 ...
1300 ...
1301 ...
1302 ...
1303 ...
1304 ...
1305 ...
1306 ...
1307 ...
1308 ...
1309 ...
1310 ...
1311 ...
1312 ...
1313 ...
1314 ...
1315 ...
1316 ...
1317 ...
1318 ...
1319 ...
1320 ...
1321 ...
1322 ...
1323 ...
1324 ...
1325 ...
1326 ...
1327 ...
1328 ...
1329 ...
1330 ...
1331 ...
1332 ...
1333 ...
1334 ...
1335 ...
1336 ...
1337 ...
1338 ...
1339 ...
1340 ...
1341 ...
1342 ...
1343 ...
1344 ...
1345 ...
1346 ...
1347 ...
1348 ...
1349 ...
1350 ...
1351 ...
1352 ...
1353 ...
1354 ...
1355 ...
1356 ...
1357 ...
1358 ...
1359 ...
1360 ...
1361 ...
1362 ...
1363 ...
1364 ...
1365 ...
1366 ...
1367 ...
1368 ...
1369 ...
1370 ...
1371 ...
1372 ...
1373 ...
1374 ...
1375 ...
1376 ...
1377 ...
1378 ...
1379 ...
1380 ...
1381 ...
1382 ...
1383 ...
1384 ...
1385 ...
1386 ...
1387 ...
1388 ...
1389 ...
1390 ...
1391 ...
1392 ...
1393 ...
1394 ...
1395 ...
1396 ...
1397 ...
1398 ...
1399 ...
1400 ...
1401 ...
1402 ...
1403 ...
1404 ...
1405 ...
1406 ...
1407 ...
1408 ...
1409 ...
1410 ...
1411 ...
1412 ...
1413 ...
1414 ...
1415 ...
1416 ...
1417 ...
1418 ...
1419 ...
1420 ...
1421 ...
1422 ...
1423 ...
1424 ...
1425 ...
1426 ...
1427 ...
1428 ...
1429 ...
1430 ...
1431 ...
1432 ...
1433 ...
1434 ...
1435 ...
1436 ...
1437 ...
1438 ...
1439 ...
1440 ...
1441 ...
1442 ...
1443 ...
1444 ...
1445 ...
1446 ...
1447 ...
1448 ...
1449 ...
1450 ...
1451 ...
1452 ...
1453 ...
1454 ...
1455 ...
1456 ...
1457 ...
1458 ...
1459 ...
1460 ...
1461 ...
1462 ...
1463 ...
1464 ...
1465 ...
1466 ...
1467 ...
1468 ...
1469 ...
1470 ...
1471 ...
1472 ...
1473 ...
1474 ...
1475 ...
1476 ...
1477 ...
1478 ...
1479 ...
1480 ...
1481 ...
1482 ...
1483 ...
1484 ...
1485 ...
1486 ...
1487 ...
1488 ...
1489 ...
1490 ...
1491 ...
1492 ...
1493 ...
1494 ...
1495 ...
1496 ...
1497 ...
1498 ...
1499 ...
1500 ...
1501 ...
1502 ...
1503 ...
1504 ...
1505 ...
1506 ...
1507 ...
1508 ...
1509 ...
1510 ...
1511 ...
1512 ...
1513 ...
1514 ...
1515 ...
1516 ...
1517 ...
1518 ...
1519 ...
1520 ...
1521 ...
1522 ...
1523 ...
1524 ...
1525 ...
1526 ...
1527 ...
1528 ...
1529 ...
1530 ...
1531 ...
1532 ...
1533 ...
1534 ...
1535 ...
1536 ...
1537 ...
1538 ...
1539 ...
1540 ...
1541 ...
1542 ...
1543 ...
1544 ...
1545 ...
1546 ...
1547 ...
1548 ...
1549 ...
1550 ...
1551 ...
1552 ...
1553 ...
1554 ...
1555 ...
1556 ...
1557 ...
1558 ...
1559 ...
1560 ...
1561 ...
1562 ...
1563 ...
1564 ...
1565 ...
1566 ...
1567 ...
1568 ...
1569 ...
1570 ...
1571 ...
1572 ...
1573 ...
1574 ...
1575 ...
1576 ...
1577 ...
1578 ...
1579 ...
1580 ...
1581 ...
1582 ...
1583 ...
1584 ...
1585 ...
1586 ...
1587 ...
1588 ...
1589 ...
1590 ...
1591 ...
1592 ...
1593 ...
1594 ...
1595 ...
1596 ...
1597 ...
1598 ...
1599 ...
1600 ...
1601 ...
1602 ...
1603 ...
1604 ...
1605 ...
1606 ...
1607 ...
1608 ...
1609 ...
1610 ...
1611 ...
1612 ...
1613 ...
1614 ...
1615 ...
1616 ...
1617 ...
1618 ...
1619 ...
1620 ...
1621 ...
1622 ...
1623 ...
1624 ...
1625 ...
1626 ...
1627 ...
1628 ...
1629 ...
1630 ...
1631 ...
1632 ...
1633 ...
1634 ...
1635 ...
1636 ...
1637 ...
1638 ...
1639 ...
1640 ...
1641 ...
1642 ...
1643 ...
1644 ...
1645 ...
1646 ...
1647 ...
1648 ...
1649 ...
1650 ...
1651 ...
1652 ...
1653 ...
1654 ...
1655 ...
1656 ...
1657 ...
1658 ...
1659 ...
1660 ...
1661 ...
1662 ...
1663 ...
1664 ...
1665 ...
1666 ...
1667 ...
1668 ...
1669 ...
1670 ...
1671 ...
1672 ...
1673 ...
1674 ...
1675 ...
1676 ...
1677 ...
1678 ...
1679 ...
1680 ...
1681 ...
1682 ...
1683 ...
1684 ...
1685 ...
1686 ...
1687 ...
1688 ...
1689 ...
1690 ...
1691 ...
1692 ...
1693 ...
1694 ...
1695 ...
1696 ...
1697 ...
1698 ...
1699 ...
1700 ...
1701 ...
1702 ...
1703 ...
1704 ...
1705 ...
1706 ...
1707 ...
1708 ...
1709 ...
1710 ...
1711 ...
1712 ...
1713 ...
1714 ...
1715 ...
1716 ...
1717 ...
1718 ...
1719 ...
1720 ...
1721 ...
1722 ...
1723 ...
1724 ...
1725 ...
1726 ...
1727 ...
1728 ...
1729 ...
1730 ...
1731 ...
1732 ...
1733 ...
1734 ...
1735 ...
1736 ...
1737 ...
1738 ...
1739 ...
1740 ...
1741 ...
1742 ...
1743 ...
1744 ...
1745 ...
1746 ...
1747 ...
1748 ...
1749 ...
1750 ...
1751 ...
1752 ...
1753 ...
1754 ...
1755 ...
1756 ...
1757 ...
1758 ...
1759 ...
1760 ...
1761 ...
1762 ...
1763 ...
1764 ...
1765 ...
1766 ...
1767 ...
1768 ...
1769 ...
1770 ...
1771 ...
1772 ...
1773 ...
1774 ...
1775 ...
1776 ...
1777 ...
1778 ...
1779 ...
1780 ...
1781 ...
1782 ...
1783 ...
1784 ...
1785 ...
1786 ...
1787 ...
1788 ...
1789 ...
1790 ...
1791 ...
1792 ...
1793 ...
1794 ...
1795 ...
1796 ...
1797 ...
1798 ...
1799 ...
1800 ...
1801 ...
1802 ...
1803 ...
1804 ...
1805 ...
1806 ...
1807 ...
1808 ...
1809 ...
1810 ...
1811 ...
1812 ...
1813 ...
1814 ...
1815 ...
1816 ...
1817 ...
1818 ...
1819 ...
1820 ...
1821 ...
1822 ...
1823 ...
1824 ...
1825 ...
1826 ...
1827 ...
1828 ...
1829 ...
1830 ...
1831 ...
1832 ...
1833 ...
1834 ...
1835 ...
1836 ...
1837 ...
1838 ...
1839 ...
1840 ...
1841 ...
1842 ...
1843 ...
1844 ...
1845 ...
1846 ...
1847 ...
1848 ...
1849 ...
1850 ...
1851 ...
1852 ...
1853 ...
1854 ...
1855 ...
1856 ...
1857 ...
1858 ...
1859 ...
1860 ...
1861 ...
1862 ...
1863 ...
1864 ...
1865 ...
1866 ...
1867 ...
1868 ...
1869 ...
1870 ...
1871 ...
1872 ...
1873 ...
1874 ...
1875 ...
1876 ...
1877 ...
1878 ...
1879 ...
1880 ...
1881 ...
1882 ...
1883 ...
1884 ...
1885 ...
1886 ...
1887 ...
1888 ...
1889 ...
1890 ...
1891 ...
1892 ...
1893 ...
1894 ...
1895 ...
1896 ...
1897 ...
1898 ...
1899 ...
1900 ...
1901 ...
1902 ...
1903 ...
1904 ...
1905 ...
1906 ...
1907 ...
1908 ...
1909 ...
1910 ...
1911 ...
1912 ...
1913 ...
1914 ...
1915 ...
1916 ...
1917 ...
1918 ...
1919 ...
1920 ...
1921 ...
1922 ...
1923 ...
1924 ...
1925 ...
1926 ...
1927 ...
1928 ...
1929 ...
1930 ...
1931 ...
1932 ...
1933 ...
1934 ...
1935 ...
1936 ...
1937 ...
1938 ...
1939 ...
1940 ...
1941 ...
1942 ...
1943 ...
1944 ...
1945 ...
1946 ...
1947 ...
1948 ...
1949 ...
1950 ...
1951 ...
1952 ...
1953 ...
1954 ...
1955 ...
1956 ...
1957 ...
1958 ...
1959 ...
1960 ...
1961 ...
1962 ...
1963 ...
1964 ...
1965 ...
1966 ...
1967 ...
1968 ...
1969 ...
1970 ...
1971 ...
1972 ...
1973 ...
1974 ...
1975 ...
1976 ...
1977 ...
1978 ...
1979 ...
1980 ...
1981 ...
1982 ...
1983 ...
1984 ...
1985 ...
1986 ...
1987 ...
1988 ...
1989 ...
1990 ...
1991 ...
1992 ...
1993 ...
1994 ...
1995 ...
1996 ...
1997 ...
1998 ...
1999 ...
2000 ...
2001 ...
2002 ...
2003 ...
2004 ...
2005 ...
2006 ...
2007 ...
2008 ...
2009 ...
2010 ...
2011 ...
2012 ...
2013 ...
2014 ...
2015 ...
2016 ...
2017 ...
2018 ...
2019 ...
2020 ...
2021 ...
2022 ...
2023 ...
2024 ...
2025 ...
2026 ...
2027 ...
2028 ...
2029 ...
2030 ...
2031 ...
2032 ...
2033 ...
2034 ...
2035 ...
2036 ...
2037 ...
2038 ...
2039 ...
2040 ...
2041 ...
2042 ...
2043 ...
2044 ...
2045 ...
2046 ...
2047 ...
2048 ...
2049 ...
2050 ...
2051 ...
2052 ...
2053 ...
2054 ...
2055 ...
2056 ...
2057 ...
2058 ...
2059 ...
2060 ...
2061 ...
2062 ...
2063 ...
2064 ...
2065 ...
2066 ...
2067 ...
2068 ...
2069 ...
2070 ...
2071 ...
2072 ...
2073 ...
2074 ...
2075 ...
2076 ...
2077 ...
2078 ...
2079 ...
2080 ...
2081 ...
2082 ...
2083 ...
2084 ...
2085 ...
2086 ...
2087 ...
2088 ...
2089 ...
2090 ...
2091 ...
2092 ...
2093 ...
2094 ...
2095 ...
2096 ...
2097 ...
2098 ...
2099 ...
2100 ...
2101 ...
2102 ...
2103 ...
2104 ...
2105 ...
2106 ...
2107 ...
2108 ...
2109 ...
2110 ...
2111 ...
2112 ...
2113 ...
2114 ...
2115 ...
2116 ...
2117 ...
2118 ...
2119 ...
2120 ...
2121 ...
2122 ...
2123 ...
2124 ...
2125 ...
2126
```